

Title

Problem-Solution Fit – SB Stocks

Problem

Beginners interested in stock trading often lack practical experience.

Learning through real financial markets can involve monetary risk, while theoretical learning alone may not clearly demonstrate portfolio management and trading operations.

User Need

Users need a platform where they can:

- Practice stock trading.
- Avoid financial risk.
- Explore stocks.
- Understand buy and sell operations.
- Manage portfolios.
- Monitor profits and losses.
- Analyze historical trends.

Proposed Solution

SB Stocks provides a virtual paper trading environment.

Users receive virtual funds and can simulate stock transactions.

The platform provides stock exploration, virtual portfolios, historical data visualization, transaction tracking, and strategy analysis.

Problem-Solution Alignment

Problem	SB Stocks Solution
Risk of financial loss	Virtual funds
Lack of practical experience	Paper trading
Difficulty tracking stocks	Stock browsing and search
Poor portfolio understanding	Virtual portfolios
Difficulty understanding P/L	Portfolio calculations
Lack of transaction visibility	Transaction history
Difficulty studying trends	Historical charts

Problem	SB Stocks Solution
Strategy uncertainty	Strategy testing

Conclusion

SB Stocks directly addresses the identified user problems by combining learning, simulation, and analysis in a single platform.

☐ Proposed Solution

Title

Proposed Solution – SB Stocks

SB Stocks is a web-based virtual stock trading and portfolio management platform.

The proposed system provides users with virtual funds that can be used to simulate stock purchases and sales.

The platform does not execute real financial transactions.

Core Solution Components

Secure User Authentication

Users can register and securely login to the application.

Stock Exploration

Users can browse and search US-listed stocks.

Market Information

The platform provides the latest available stock information and historical market data.

Virtual Trading

Users can simulate buy and sell transactions using virtual funds.

Portfolio Management

Users can create and manage multiple portfolios.

Performance Tracking

The platform calculates portfolio values and profit/loss.

Transaction History

Every trading operation is recorded.

Strategy Analysis

Historical data can be used to evaluate basic trading strategies.

Administrative Monitoring

Admins can monitor users, orders, and transactions.

Benefits

- No financial risk.
- Practical trading experience.
- Centralized portfolio tracking.
- Historical stock analysis.
- Improved understanding of trading workflows.
- Beginner-friendly learning environment.

Educational Disclaimer

SB Stocks is designed for educational and simulation purposes only. It does not provide financial advice or execute real stock market transactions.

☐ **Solution Architecture**

Title

Solution Architecture – SB Stocks

Architecture Overview

SB Stocks uses a three-tier full-stack architecture consisting of:

1. Frontend Layer.
2. Backend Layer.
3. Database Layer.

Frontend Layer

The frontend is developed using React and Vite.

Frontend modules include:

- Landing Page.
- Login.
- Registration.
- User Dashboard.
- Stock Listings.
- Stock Charts.
- Portfolio.
- Trading Interface.
- History.
- Profile.
- Admin Dashboard.

The frontend communicates with the backend using HTTP REST API requests.

Backend Layer

The backend is developed using Node.js and Express.js.

The backend contains APIs for:

- Users.
- Authentication.
- Stocks.
- Portfolios.
- Transactions.
- Orders.
- Administration.
- Strategy Analysis.

The backend follows the MVC architectural pattern.

Model Layer

Defines MongoDB schemas using Mongoose.

Controller Layer

Contains application business logic.

Route Layer

Maps HTTP requests to controller functions.

Middleware Layer

Handles authentication and authorization.

Database Layer

MongoDB stores:

- Users.
- Stocks.
- Portfolios.
- Transactions.
- Orders.

Data Flow

User Interface

↓

React Frontend

↓

Axios HTTP Request

↓

Express REST API

↓

Authentication Middleware

↓

Controller

↓

Mongoose Model

↓

MongoDB Database

↓

API Response

↓

React User Interface

Security Architecture

JWT authentication protects user APIs.

Passwords are stored using secure hashing.

Admin APIs require role-based authorization.

Important trading calculations are performed by the backend.